Users, Requirements, and Constraints

Software Engineering

Enough shared understanding to build something useful

Gregory S. DeLozier, PhD

Enough understanding to begin

“The perfect is the enemy of the good.”

Agree on a useful next step.

Keep open questions visible.

Who depends on the result?

Person	What matters to them
Volunteer	Am I confirmed?
Organizer	Which shifts are uncovered?
Treasurer	What will this cost?
Next year’s officers	Can we keep it running?

Watch the work

Approach	Starting point
Ask about an actual event	“Tell me about the last cancellation.”
Watch the work	“Show me how you update the schedule.”
Try a rough sketch	“Where do I approve them?”

What did the first description leave out?

A user story connects need and purpose

> As a volunteer, I want to see which shifts still need help so I can choose one where I am useful.

Role	Capability	Reason
Volunteer	See unfilled places	Choose where help is needed

Describe a useful experience.

A story card leaves room to learn

Find a shift that needs help	Details
Example	Saturday, 9-10 a.m.: two unfilled places
Open question	Do pending requests count against capacity?
Proposed check	The schedule shows two unfilled places

A short record supports the conversation.

From a story to an observable result

Alex's action	Result under the approval proposal
View the schedule	Two unfilled places
Request the shift	Saved; awaiting review
Check after approval	Confirmed date and time

On paper, write a story card for checking approval status.

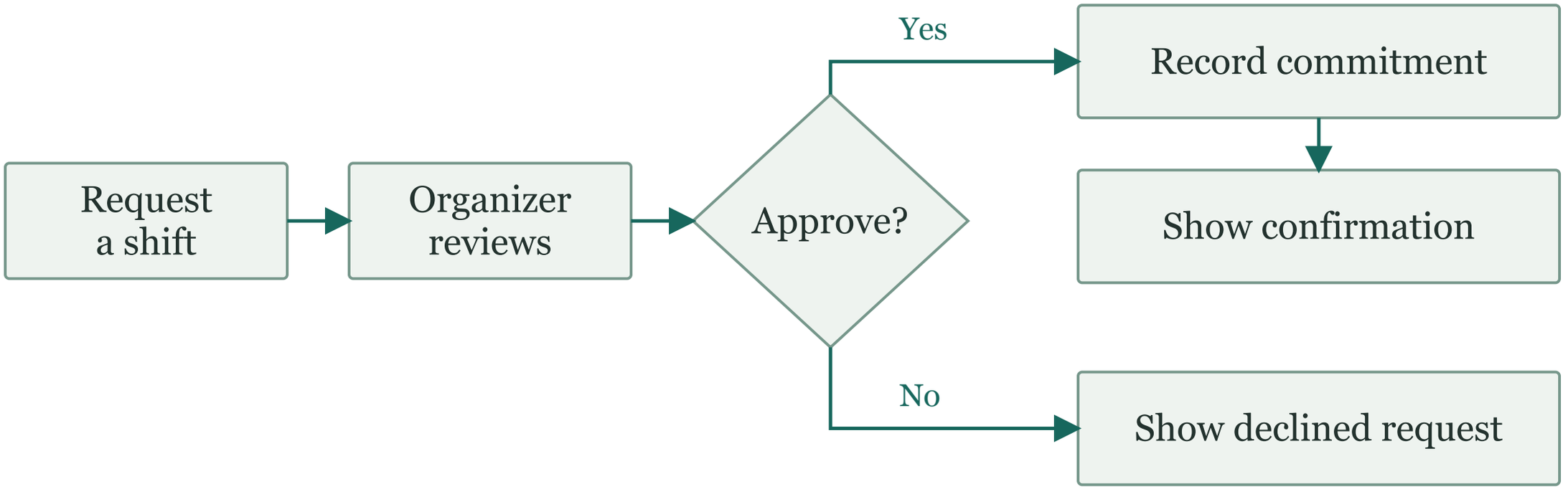
Include **role, capability, reason, one observable result, and one open question**.

Need, requirement, design choice

Level	Example
Need	“I need to know I’m on the schedule.”
Requirement	View my confirmed event and shift time
Design choice	A green checkmark beside my name

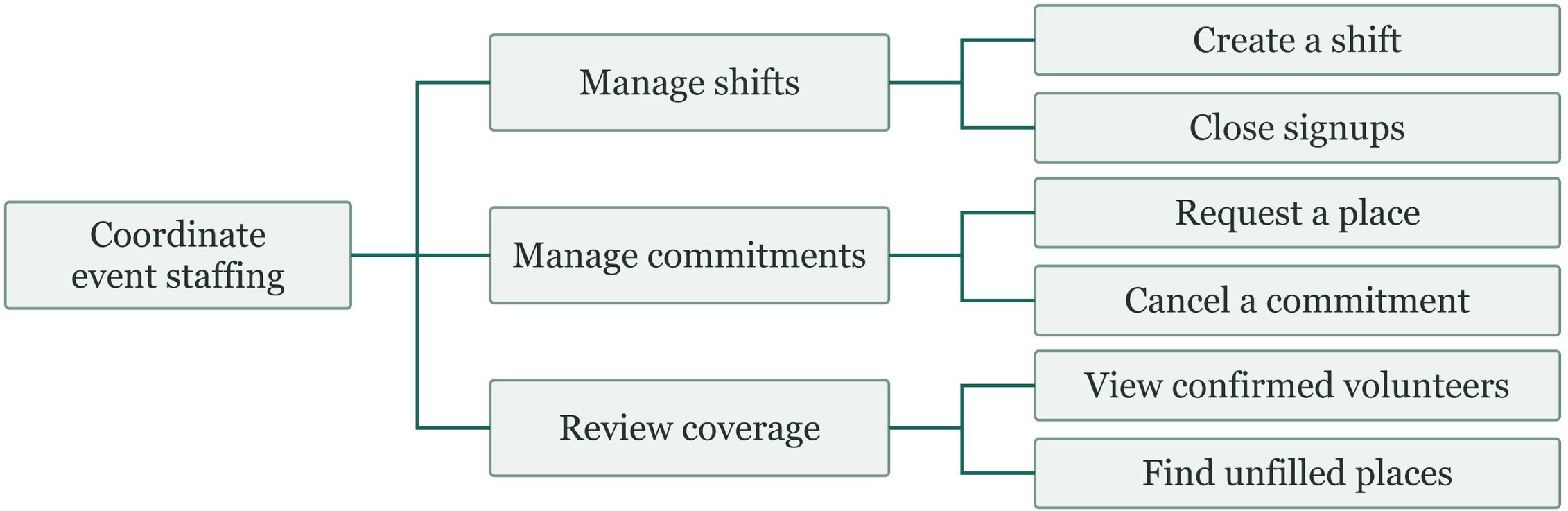
What does the checkmark promise?

A proposed signup workflow



Does a request reserve a place?

A function hierarchy



Lines mean “includes,” not “happens next.”

What makes it a car?

Basic function	Safety constraint
Move people from place to place	Must not catch fire in a collision at parking-lot speed

It must be a car, but it also must be an acceptable car.

Children's pajamas

> Must not be flammable.

Functional or nonfunctional?

Important or not important?

Function and constraint

Product	Function	Constraint
Candy bar	Food to eat	No mold
Nerf gun	Fire foam darts	Below a specified velocity
Rocket	Carry a payload into flight	No toxic fuel dropped on the ground

Performing the function is only part of meeting the requirements.

Make quality concrete

Vague request	A useful question
Reliable	What survives a restart?
Private	Who may see contact details?
Easy to use	Can a new member sign up without help?

Find out what acceptable means here.

Say exactly what you mean

> “Automatically notify everyone whenever anything changes.”

A proposal: when a confirmed volunteer cancels, email the event organizer immediately.

Who needs to know what, and when?

When needs conflict

Volunteer	Organizer
“My plans can change.”	“I need dependable coverage.”

What should happen to a late cancellation?

What can help before the event?

The event is in **ten days**.

The budget is **zero**.

Which useful part can be ready?

Checking the result

Validation	Verification
Does this solve the actual problem?	Does it behave as specified?
Volunteers know whether they are confirmed	Cancellation follows the agreed rule

A correctly built request form can still leave everyone uncertain.

Try the cancellation question

> “People should be able to cancel, but I can’t have everyone disappearing just before the event.”

Propose a rule. Mark it as a proposal.

What example would reveal a disagreement?

Make the next commitment clear

Agree on what the next useful piece should do.

Record the questions that remain open.

Leave people a fair chance to finish work they can be proud of.